

Research

SQUMUTH squirrel search based algorithm for high order mutant generation in mutation testing

Subhasish Mohanty¹ · Jyotirmaya Mishra¹ · Sudhir Kumar Mohapatra² · Seifu Detso Bejo³ · Aliazar Deneke Deferisha⁴

Received: 8 June 2024 / Accepted: 27 March 2025

Published online: 01 May 2025

© The Author(s) 2025 **OPEN**

Abstract

In today's software testing community, quality assessment remains critical, with mutation testing standing as a cornerstone technique for evaluating the effectiveness of test cases. This method involves introducing faulty code, or mutants, into the program to assess the quality of the test suite and other testing methods. However, mutation testing faces challenges such as the generation of numerous mutants, the presence of equivalent mutants that are difficult to detect through testing, and the lack of realistic mutant creation. Literature reviews indicate significant efforts to address these issues through formal solutions and heuristic methods. Recent optimization-based methods are now recognized as cost-effective result in an optimized solution. Hence, to overcome these limitations, this study introduces SQUMUTH, a novel approach for high-order mutant generation based on the Squirrel Search Algorithm (SSA). Inspired by the foraging behavior of squirrels, SSA offers a promising solution for enhancing the efficiency and effectiveness of mutation testing. Experimental evaluations on eight well-known Java benchmark programs demonstrate that SQUMUTH outperforms existing methods. Comparative analyses of mutation scores and the rates of realistic mutants consistently show its better performance across all subject programs compared to other state-of-the-art methods such as Social Group Optimization, Binary Genetic Algorithm, and random testing. The experimental results underscore its effectiveness in generating more realistic mutants. The experimental results indicated that the proposed approach has the potential to advance software testing by improving the cost-effectiveness of mutation analysis and the quality of software systems.

Keywords Software testing · Mutation testing · Squirrel search algorithm · First-order mutants · Higher-order mutants · Mutation score

1 Introduction

In the present era of technological advancement, a wide array of software systems plays vital roles in our everyday existence, ranging from mobile applications to complex enterprise systems [1]. These systems require the highest level of quality, as any faults within them can lead to significant problems. Achieving such high-quality software is a challenging endeavor that requires a rigorous testing approach. Generating effective test cases are crucial for

✉ Sudhir Kumar Mohapatra, sudhir.mohapatra@srisriuniversity.edu.in; Subhasish Mohanty, subhasish2307@gmail.com; Jyotirmaya Mishra, jyotirmayamishra75@gmail.com; Seifu Detso Bejo, seifu.detso@wku.edu.et; Aliazar Deneke Deferisha, aliazar.deneke@amu.edu.et | ¹Computer Science and Engineering, GIET University, Gunupur, Odisha, India. ²Faculty of Engineering and Technology, Sri Sri University, Cuttack, India. ³Department of Software Engineering, CCI, Wolkite University, Wolkite, Ethiopia. ⁴Faculty of Computing and Software Engineering, Arba Minch University, Arba Minch, Ethiopia.



identifying potential faults and ensuring comprehensive software coverage. High-quality test cases not only verify that the software meets its requirements but also uncover subtle bugs that could compromise reliability and user satisfaction [2, 3].

Mutation testing (MT) is one of the powerful testing techniques used to assess the effectiveness of test cases by inserting faulty code, called mutants, into the program being tested. This process helps to determine if the existing test cases can detect these mutants, and the quality of the mutants is vital for measuring the effectiveness of MT [4]. When MT is effective, it ensures software quality by assessing the effectiveness of test suites and can also be used to evaluate the effectiveness of other testing techniques. Despite its advantages, MT has certain limitations that make it complex, computationally expensive, and less popular as a testing technique. These limitations include the generation of large number of first order mutants (FOMs), the presence of equivalent mutants (EQs), which are difficult to detect by the given test suite, and the difficulty of creating realistic mutants that accurately represent realistic software faults [5].

To improve the effectiveness of MT by addressing these limitations, different researchers have proposed various techniques based on weak mutants, second-order mutants (SOMs), and higher-order mutants (HOMs). Among them, HOM based techniques [6] have emerged as a very promising solution because it involves combining FOMs to create more complex mutants, which leads to a reduction in the number of mutant generations, minimizes the occurrence of EQs, and produces mutants that closely resemble realistic software faults.

However, generating HOMs presents an optimization challenge that necessitates maximizing fault detection, optimizing mutations, balancing effectiveness and efficiency, exploring trade-offs, and efficiently navigating the solution space [7]. To address these optimization challenges, various search-based software testing (SBST) techniques, which are nature-inspired meta-heuristic algorithms, have been utilized. In the software testing literature, the most prominently applied techniques for SBST include Particle Swarm Optimization (PSO) [8], Genetic Algorithm (GA) [9], Harmony Search [10], Hill Climbing [11], Social Group Optimization (SGO) [12], and the Bat Algorithm [13]. These techniques, along with others, have proven to be suitable for achieving cost-effective results in an optimized solution by offering systematic and efficient methods for exploring the solution space, prioritizing valuable mutants, and balancing the trade-offs involved in HOM generation [14].

However, while the state-of-the-art approaches have shown promising results, further improvements are needed in optimization, accuracy, convergence rate, and computational efficiency. To bridge these gaps, we introduce SQU-MUTH, a novel approach for HOM generation in MT. The proposed strategy utilizes the Squirrel Search Algorithm (SSA), a nature-inspired meta-heuristic optimization technique. SSA has demonstrated remarkable performance compared to other renowned optimization algorithms, particularly in terms of convergence speed and optimization accuracy [15]. The SSA offers several advantages over other metaheuristic algorithms, making it a powerful optimization technique. Its adaptive exploration–exploitation mechanism, inspired by the dynamic foraging behavior of squirrels, guarantees faster convergence and prevents premature stagnation in local optima. Unlike traditional algorithms such as PSO, GA, or ACO, SSA maintains a better balance between diversification and intensification through seasonal monitoring and varied movement strategies like gliding, jumping, and switching between different food sources. This dynamic adaptability allows SSA to perform effectively in high-dimensional and multi-modal optimization problems. Moreover, SSA requires fewer parameters to tune, making it easy to implement while still demonstrating superior robustness in handling complex search spaces. Its ability to escape local optima and adapt to changing conditions makes it highly suitable for applications such as engineering design, machine learning hyperparameter tuning, software defect prediction, and medical disease classification. Compared to other metaheuristics, SSA provides a well-structured and efficient search process, making it a strong candidate for solving real-world optimization problems. By applying SSA, we aim to overcome the limitations of existing HOM generation methods, providing a more efficient and effective solution.

This research study makes the following contributions:

1. Proposed a novel approach, SQU-MUTH, which is based on the nature-inspired meta-heuristic SSA for HOM generation. To the best of the authors' knowledge, we are the first to model SSA for generating HOMs in MT.
2. Developed SQU-MUTH Prototype and Analysis: The authors implemented SQU-MUTH using Java programming language. We conducted empirical and theoretical analysis to evaluate SQU-MUTH's effectiveness in fault detection capability and quality of the generated mutants.
3. Outperforming State-of-the-Art Methods: Experimental results demonstrate that SQU-MUTH outperforms across all eight subject programs compared to alternative approaches such as SGO, Binary Genetic Algorithm (BGA), and random testing.

The remaining sections of the study are structured as follows: Sect. 2 provides a concise overview of the related works. The problem description of HOM generation is discussed in Sect. 3. Section 4 introduces the proposed solution. The results and discussions related to the proposed solution are presented in Sect. 5. Threats to validity are addressed in Sect. 6. Finally, Sect. 7 outlines the conclusion and future work.

2 Literature review

In software testing, MT is essential for assessing test suite robustness but presents significant challenges. It struggles with redundant FOMs, the occurrence of EQs, which are undetectable artificial faults, and creating unrealistic HOMs that fail to reflect actual software faults [5, 16]. These drawbacks make MT complex and computationally expensive. Under consideration of these limitations, various methods have been proposed to generate quality test cases. This review examines MT comprehensively, focusing on HOM generation techniques, their strengths, limitations, and ongoing research addressing these challenges. While no universal solution exists, numerous studies contribute to this evolving field [17]. To structure the analysis, literature is categorized by methodological approaches, including machine learning (ML)-based techniques, methodology and evolutionary algorithms, hybrid approaches, context-aware methods, and domain-specific strategies.

2.1 Machine learning-based approaches

In recent years, ML techniques have been explored to improve the effectiveness of MT by optimizing mutant selection and reducing the number of redundant test cases. Ramu and Akila [18] proposed a ML based method for classifying effective mutants to reduce the computational cost of MT for smart contracts. By comparing various classifiers, they aim to distinguish between effective and useless mutants, thereby optimizing the MT process for smart contracts deployed on blockchain platforms. Moradi Dakhela et al. [19] proposed MuTAP, an approach utilizing MT to enhance test case generation by Large Language Models (LLMs). By augmenting prompts with surviving mutants, MuTAP improves bug detection, achieving a 93.57% Mutation Score and detecting 28% more faulty code than existing methods. Tambon et al. [20] introduced a Probabilistic Mutation Testing (PMT) approach for Deep Neural Networks (DNNs) to mitigate inconsistencies from stochastic training. PMT enhances reliability and ensures consistent mutant evaluation, proving effective across three models and eight mutation operators.

2.2 Metaheuristic and evolutionary approaches

Metaheuristic algorithms have been widely applied in MT to optimize HOM generation by improving fault detection and test case effectiveness. Langdon and Clark [19] investigated the use of genetic improvement techniques for generating mutants in deeply nested parallel computing benchmarks using the MAGPIE framework. Their study revealed that a significant portion of the mutants produced over 53% identical results to the original program, indicating limited disruption propagation and reduced effectiveness in fault detection. This highlights a fundamental challenge in search-based MT, where a lack of targeted selection often leads to the generation of redundant mutants. Abuljadayel et al. [22] introduced a method utilizing GA to enhance the generation of HOMs in MT. Evaluated on Java programs, this approach demonstrated a significantly higher defect detection rate compared to traditional MT techniques. However, GA-based methods often generate excessive numbers of mutants without effectively distinguishing between impactful and non-impactful changes, leading to increased computational complexity.

Wedyan et al. [23] introduced GaSubtle, a GA-based approach aimed at producing subtle HOMs. GaSubtle initiates by generating a population of HOMs randomly, which are then assessed using test cases. The mutation process continues until a predetermined number of generations is reached. The efficacy of GaSubtle on Java programs outperforms that of traditional MT techniques. Mohanty et al. [24] presented an approach for generating HOMs using a PSO algorithm with Lévy flight (LFPSO). Traditional HOM generation techniques often overlook the contextual aspects during mutant creation, thereby limiting their ability to detect defects. To overcome this constraint, the authors implemented LFPSO, resulting in improved effectiveness in generating HOMs on Java programs than traditional MT techniques in terms of defect detection.

2.3 Hybrid approaches combining techniques

Hybrid methods integrate multiple optimization techniques to enhance the efficiency of HOM generation. Lemieux et al. [25] introduced CODAMOSA, an algorithm combining SBST with LLMs like Codex to escape coverage plateaus. When SBST stalls, CODAMOSA uses LLM-generated test cases to explore under-covered functions, leading to significantly higher coverage across 486 benchmarks compared to traditional SBST. Detso et al. [26] proposed a hybrid approach called BackIP, which combines the Backtracking Search Optimization (BSA) algorithm with the Different Operator Algorithm (DOA). Integrated with the MjJava mutation system, BackIP improved cost-effectiveness and test suite reduction by outperforming the state-of-the-art techniques like PSO-MT, GA-MT, SGO-MT, and ABC-MT on benchmark programs. However, its applicability is limited to specific data type benchmark programs, reducing its effectiveness in real-world software systems.

Wong et al. [27] presented a method for identifying HOMs using variational execution. They employed variational execution to explore different combinations of FOMs, which are created by mutating multiple statements in a program. The efficacy of an approach evaluated on benchmark programs performed better in terms of defect detection compared to traditional MT techniques. Oh et al. [28] proposed an approach for sampling HOMs using causal effect analysis. By considering the causal relationships between statements in a program, their approach can identify mutants that are more likely to cause defects. The approach performs better than traditional HOM sampling techniques in defect detection and identification.

2.4 Context-aware and domain-specific approaches

Several studies have focused on developing context-aware and domain-specific MT techniques tailored to specific programming languages and application areas. Ojdanic et al. [29] examined the correlation between syntactic and semantic similarity in MT, highlighting the limitations of existing fault-seeding techniques in capturing real-world software defects. The analysis of four fault-seeding techniques revealed varying degrees of semantic resemblance to real faults, with PiTest, IBIR, μ BERT, and DeepMutation showing 65.11%, 76.44%, 61.39%, and 9.76% similarity, respectively. Their findings provide valuable insights into mutation analysis but do not propose an explicit method for improving HOM generation.

Van Nho et al. [30] proposed an approach aimed at enhancing the quality of mutation operators and the effectiveness of test cases. This was achieved by employing Higher Order Mutation Testing (HOMT) algorithms such as LastToFirst and NeighPair. The evaluation results indicate that the proposed method identifies more defects compared to traditional HOMT techniques on known benchmark programs. Van Phol et al. [31] proposed a Lustre-specific approach for HOMs in the Lustre language that are used for reactive, control, and monitoring systems. Their technique produces realistic mutants that align with the characteristics of Lustre-based applications, enhancing fault detection in embedded systems. However, the approach is limited to Lustre programs, restricting its generalizability to mainstream programming languages such as Java and C++ .The summary of related work is presented in Table 1.

In general, drawing from the literature review, various approaches have been proposed to enhance the efficiency and effectiveness of MT, each offering strengths such as improved fault detection, enhanced coverage, and better cost-effectiveness. Despite these advancements, no universal solution has emerged, highlighting the need for further research due to key challenges, including high computational overhead, unrealistic fault representation, excessive EQs, benchmark dependency, and scalability issues. Many methods suffer from mutant explosion and high resource consumption [18, 19, 21–23]. Additionally, search-based approaches often fail to generate semantically meaningful HOMs, limiting fault detection effectiveness [24, 29, 30]. The persistence of redundant and EQs further increases computational costs, as many methods lack proper filtering mechanisms [21–23, 25]. Furthermore, several approaches are benchmark-specific, restricting generalizability to diverse software environments [20, 26, 27, 31]. Scalability remains a bottleneck, as most techniques struggle with large-scale applications due to increasing test execution complexity [21–23, 28].

To address current challenges, metaheuristic optimization techniques offer potential solutions, promising more efficient and comprehensive HOM generation. In response, the authors proposed the SSA-based SQUMUTH approach, which boasts improved convergence rates, accuracy, and scalability over other algorithms. The rationale behind the selection of SSA is to apply its exploration and exploitation principles, which enhance the efficacy of the MT approach. Unlike GA- and PSO-based methods, SSA enables adaptive exploration and exploitation, optimizing mutant selection and reducing computational overhead. SQUMUTH further enhances fault representation using Realistic Mutant Rate (RMR)

Table 1 Tabular summary of related works

Reference	Approach	Key contribution	Strengths	Limitations	Classification
Ramu and Akila [18]	ML-based Mutation Classification	Classifies effective mutants to reduce computational costs in MT for smart contracts	Optimizes MT for blockchain platforms	Focused on smart contracts, may not generalize to other domains	Machine learning
Moradi Dakhela et al. [19]	MuTAP (MT Augmented Prompting)	Enhances test case generation using LLMs and surviving mutants, achieving a 93.57% Mutation Score	High Mutation Score, improves bug detection	Focused on LLMs, limited to specific applications	Machine learning
Tambon et al. [20]	PMT	Addresses inconsistencies in DNNs caused by stochastic training	Improves reliability over traditional methods	Specific to DNNs, it may not generalize to other areas	Machine learning
Langdon and Clark[21]	MAGPIE (Genetic Improvement)	Evaluates genetic improvement on deeply nested parallel computing benchmarks	Highlights challenges in deeply nested code	Limited disruption propagation may not affect external behavior	Metaheuristic
Abuljadayel et al. [22]	GA for HOMs	Uses GA to generate HOMs with higher defect detection rates	Higher defect detection rate	Limited to Java programs	Metaheuristic
Wedyan et al. [23]	GaSubtle (GA for Subtle HOMs)	Produces subtle HOMs using GA, outperforming traditional MT	Higher efficacy produces subtle mutants	Complexity in mutation process	Metaheuristic
Mohanty et al. [24]	PSO with Lévy Flight (LFPSO)	Generates HOMs using LFPSO, improving defect detection	Outperforms traditional MT techniques	Complexity in integrating PSO with LFPSO	Metaheuristic
Lemieux et al. [25]	CODAMOSA	Combines SBST with LLMs like Codex to escape coverage plateaus	Significant improvement in coverage	Dependent on LLM effectiveness	Hybrid (SBST + LLM)
Bejo et al. [26]	BackIP (Hybrid Approach)	Combines BSA and DOA with Mujava for improved cost-effectiveness	Outperforms state-of-the-art techniques, reduces test suite sizes	Limited to specific benchmark programs	Hybrid (BSA + DOA)
Wong et al. [27]	Variational Execution	Identifies HOMs by exploring different combinations of FOMs	Better defect detection	Specific to Java programs	Hybrid (variational execution)
Oh et al. [28]	Causal Effect Analysis for HOM Sampling	Identifies mutants likely to cause defects by analyzing causal relationships	Improved defect detection	Limited to causal effect analysis	Hybrid (causal analysis)
Ojdanic et al. [29]	Syntactic vs. Semantic Similarity	Investigates correlation between syntactic and semantic similarity in MT	Provides insight into varying degrees of semantic resemblance	Focused on specific fault-seeding techniques	Fault seeding analysis
Van Nho et al. [30]	HOMT Algorithms (LastToFirst, NeighPair)	Enhances mutation operators' quality and test case effectiveness	Identifies more defects than traditional techniques	Dependent on specific benchmark programs	Context-aware HOMT
Van Phol et al. [31]	Lustre-Specific HOM Generation	Produces more realistic mutants in Lustre-based reactive, control, and monitoring systems	Enhances error detection in Lustre-specific systems	Limited to Lustre language	Domain-specific (Lustre)

categorization, ensuring semantically richer HOMs. Its intelligent replacement strategy minimizes redundant and EQs, improving test effectiveness. By prioritizing impactful mutations, SQUMUTH ensures computational feasibility even for large-scale software systems, making it a robust and scalable solution for modern MT.

3 Higher order mutants

The HOM generation problem presents a set of interconnected challenges and opportunities within MT. Traditional MT typically involves generating FOMs, where a single, simple syntactic change is made to the program under test, which led to the problem of mutant explosion, complicating the testing process and significantly increasing the computational cost [32]. As software systems grow in complexity, the limitations of focusing solely on FOMs become apparent, driving the need for more complex testing strategies, such as HOMT. HOMT [33] has emerged as a significant evolution in the field of MT by generating HOMs, addressing some of the critical challenges posed by traditional MT. One of the primary challenges in traditional MT is the large number of mutants generated, known as the mutant explosion problem [34]. With FOMs, each representing a unique alteration, the number of mutants can quickly become unmanageable, making the testing process both time-consuming and computationally demanding. Extending this concept to HOMs, which involve combinations of multiple FOMs, intensifies this issue. The exponential increase in potential mutants requires more sophisticated strategies to manage and analyze these mutants effectively.

In addition to mutant explosion problem, traditional MT has been criticized for its lack of realism in fault representation introduced by FOMs [35]. Because FOMs represent isolated, simple changes, they may not accurately reflect the more complex and subtle faults found in real-world software. This lack of realism can result in a test process that fails to uncover critical bugs, ultimately undermining the reliability of the software. HOMT addresses this by enabling the creation of HOMs, which combine multiple FOMs to simulate more complex fault scenarios. These HOMs have the potential to reveal subtle and complex faults that would otherwise go undetected, thereby improving the overall robustness of the software testing process.

Another significant challenge in traditional MT is the generation of EQs, where mutants that, despite the changes made, behave identically to the original program for all test cases [36]. These mutants do not contribute to the effectiveness of the test suite and require considerable effort to identify and eliminate. HOMT offers a solution by potentially reducing the occurrence of EQs. By carefully selecting and combining FOMs in ways that lead to more realistic and distinct fault scenarios, HOMT can minimize the generation of EQs that are semantically similar to the original program, thus improving the efficiency of the testing process. Besides, HOMT introduces the concept of subsuming HOMs [37], where mutants that are more difficult to kill than their constituent FOMs. These subsuming HOMs push the testing process further, ensuring that test cases are not only effective at detecting simple faults but are also robust enough to uncover more complex issues. This approach directly addresses the need for more rigorous testing in increasingly complex software environments.

However, the practical implementation of HOMT is not without its challenges. While HOMT can reduce the overall number of mutants by focusing on more complex and meaningful combinations, the process of generating and testing HOMs requires significant computational resources. This necessitates the development of efficient algorithms and strategies, such as the proposed SQUMUTH approach in this research, to make HOMT a viable and effective tool for modern software testing. Ultimately, the HOM generation problem is a complex challenge that involves balancing trade-offs between test effectiveness, computational cost, the realism of fault representation, and minimizing the occurrence of equivalent mutants, all of which are critical to improving the reliability and robustness of the testing approach and software systems.

4 The proposed solution

The proposed SQUMUTH method focuses on generating HOMs by introducing a novel algorithm called SSA. This algorithm has been specifically designed to overcome the limitations found in existing algorithms by introducing innovative features and mechanisms. The SSA [15] is a swarm intelligence-based, nature-inspired metaheuristic algorithm that draws inspiration from the behavior of flying squirrels. Flying squirrels are known for their ability to glide from tree to tree, using adaptive strategies to find food and avoid predators. SSA utilizes this behavior for optimization, representing a population of squirrels as candidate solutions searching for the best solution in a defined problem space.

The approach aims to achieve required coverage for mutation analysis by executing a set of HOMs (M) and a test suite (TC). The test suite consists of a collection of test cases (TC_d), labeled as $\{tc_1, tc_2, tc_3 \dots tc_d\}$, each representing a specific scenario to be tested. The set of HOMs (M), denoted as $\{M_1, M_2, M_3 \dots M_n\}$, is created from the original program through a series of mutation operators and is crucial for evaluating the effectiveness of the test cases. The main objective is to ensure that each test case (tc_j) in the test suite (TC_d) satisfies the strong mutant for M_i . This criterion guarantees that all test cases adequately cover the strong mutant and provide the desired coverage for mutation analysis. The design of SQUMUTH encompasses the following steps while aligning key components of the algorithm and problem domain.

4.1 Initialization

The design starts by defining input parameters: that includes the number of iterations ($iter_max$), the total number of mutants or squirrels (N), the decision variable or the size of test suite (D), the lower bound (M_L), the upper bound (M_U) of the i th mutants in the j th dimension, $U(0, 1)$ which represents a random number uniformly distributed in the range $[0, 1]$, the gliding distance (dg), the gliding constant (Gc), and the random distributions $R1$, $R2$, and $R3$ in the range $[0, 1]$, the predator presence probability (Pdp), the loss in height after gliding (hg), which is used to calculate dg , and the scaling factor (sf).

First, to generate randomly an initial input of set of mutants (M_i) that defines how the mutant candidates represent the flying squirrels (M) in the SSA using (Eq. (1)). This equation ensures that mutants are distributed across the defined mutation space, enabling a diverse search process.

$$M_i = M_L + U(0, 1) \times (M_U - M_L). \quad (1)$$

To systematically evaluate SQUMUTH, the authors constructed a mutant detection matrix (Eq. (2)), assigning values: 1 for mutants killed by test cases, 0 for mutants not killed by specific test cases and -1 for equivalent mutants not killed by the entire test suite. This matrix provides an efficient way to assess mutant effectiveness, filtering out redundant mutants and enhancing mutation analysis.

$$M_{ij} = \begin{bmatrix} M_{11} & M_{12} & M_{13} & \dots & M_{nd} \\ M_{21} & M_{22} & M_{23} & \dots & M_{nd} \\ M_{31} & M_{32} & M_{33} & \dots & M_{nd} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ \vdots & \vdots & \vdots & \dots & \vdots \\ \vdots & \vdots & \vdots & \dots & \vdots \\ M_{n1} & M_{n2} & M_{n3} & \dots & M_{nd} \end{bmatrix}. \quad (2)$$

4.2 Fitness evaluation and categorization of mutants

The fitness of each mutant is evaluated based on its performance against test cases, using a fitness function (Eq. 3) with values ranging from 0 to 1. A value of 0 suggests that no test case can kill the mutant, indicating it might be an equivalent mutant. As the fitness value increases, the mutant weakens, with a value of 1 signifying that any test case can kill it.

$$\text{Fitness}(M) = \frac{\text{test cases that killed } M}{\text{Test cases in the TC}}. \quad (3)$$

To measure HOM effectiveness, the RMR is calculated using (Eq. (4)). The RMR of a HOM is the ratio of its fitness to the combined fitness of its FOMs. If RMR exceeds 1, the HOM is weaker than its FOMs, while lower values suggest the HOM is stronger.

$$\text{RMR} = \frac{\text{fitness}(\text{HOM})}{\text{fitness}(\text{FOM1}) + \text{fitness}(\text{FOM2}) \dots \text{fitness}(\text{FOMn})}. \quad (4)$$

Mutants are sorted by ascending RMR for efficient categorization. This categorization helps prioritize more effective HOMs, reducing redundant and equivalent mutants while improving mutation testing efficiency. To align with SSA's foraging analogy, mutants are categorized into three groups based on RMR values as presented in Table 2. To address the optimization problems, the SSA algorithm incorporates three types of trees: hickory nut trees with optimal solutions,

Table 2 Mutant categorization based on RMR and its impact on MT

Mutant category	RMR Interval	SSA tree assignment	MT Impact
Strong mutants	0.0–0.5	Hickory nut trees (optimal)	Highly useful for robust test evaluation
Intermediate mutants	0.5–1.0	Acorn nut trees (moderate)	Balanced mutants, useful for progressive testing
Weak mutants	> 1.0	Normal nut trees (poor)	Least effective; replaced with stronger HOMs

acorn nut trees that are expected to trend towards hickory trees, and normal nut trees. In line with this analogy, the authors of the study classified the candidate solution of HOMs into three categories based on their RMR value intervals: strong mutants (0.0–0.5), intermediate mutants (0.5–1.0), and weak mutants (> 1.0).

Some squirrels are known to randomly prefer hickory trees once they have enough energy, while others utilize acorn nut trees. The foraging patterns of flying squirrels are influenced by predators, which are simulated in this study by updating locations while considering the probability of encountering a predator (P_{dp}). Then declare or assign strong mutants on the hickory nut tree, intermediate mutants on the acorn nut tree, and weak mutants on normal trees. After categorizing the mutants to maintain diversity and explore the search space, the new HOMs will be generated to replace the weak mutants. In HOMT, mutation operators are specifically designed to generate HOMs by focusing on constructs such as loops, conditionals, and function calls.

4.3 Optimization and movement strategy

SQUMUTH optimizes mutant selection through SSA-inspired movement strategies: for exhibits their dynamic food-search behavior, which gives rise to three distinct scenarios. In each scenario, it is assumed that without the presence of predators, the flying squirrel effortlessly glides through the forest, successfully locating its favored food. However, when predators are present, the squirrel must resort to random walking to search for a nearby hiding location. To simulate the movement of flying squirrels while considering the replaced weak mutants, the three scenarios presented as follows.

Scenario 1: Intermediate mutants (M_{at}) move to strong mutant positions determined using (Eq. (5))

$$M_{at}^{t+1} = \begin{cases} M_{at}^t + d_g + G_c \times (M_{ht}^t - M_{at}^t) & R_1 \geq P_{dp} \\ \text{Random location} & \text{otherwise} \end{cases} \quad (5)$$

Scenario 2: Weak mutants (M_{nt}) move to Intermediate mutant positions determined using (Eq. (6))

$$M_{nt}^{t+1} = \begin{cases} M_{nt}^t + d_g + G_c \times (M_{at}^t - M_{nt}^t) & R_2 \geq P_{dp} \\ \text{Random location} & \text{otherwise} \end{cases} \quad (6)$$

Scenario 3: Weak mutants (M_{nt}) are completely replaced with stronger HOMs determined using (Eq. (7))

$$M_{nt}^{t+1} = \begin{cases} M_{nt}^t + d_g + G_c \times (M_{ht}^t - M_{nt}^t) & R_3 \geq P_{dp} \\ \text{Random location} & \text{otherwise} \end{cases} \quad (7)$$

This dynamic approach ensures better mutant selection, balancing exploration and exploitation in SSA. In all scenarios, R_1 , R_2 , and R_3 are random numbers between [0, 1]. G_c represents the gliding constant, set to 1.9 that maintains a balance between exploration and exploitation. P_{dp} is set to 0.1, and d_g represents the random gliding distance.

4.4 Seasonal monitoring and termination criteria

To prevent early convergence, SSA integrates seasonal monitoring, using Levy distribution (Eq. (8)) to diversify mutant selection, helping to find new better solutions. Adjust the seasonal constant based on the progress of mutant generation to adapt the search strategy. This is achieved by checking the seasonal monitoring condition ($S_c^t < S_{min}$) using Eqs. (9) and (10). In this context, 't' represents the current iteration value, and 'tm' denotes the maximum iteration value. The value of S_{min} impacts the exploration and exploitation abilities of SSA.

Table 3 SSA parameter setting

Parameter setting	Value
iter_max	100
M_L	0
M_U	1
U	0–1
d_g	0.1
G_c	1.9
R1, R2 and R3	0–1
P_{dp}	0.1
h_g	8 m
S_f	18

$$M_{nt}^{new} = ML + \text{Lévy}(n) \times (MU - ML) \quad (8)$$

$$S_c^t = \sqrt{\sum_{k=1}^d (M_{at,k}^t - M_{ht,k}^t)^2} \quad (9)$$

$$S_{min} = \frac{10E^{-6}}{(365)^{t/(\frac{tm}{25})}} \quad (10)$$

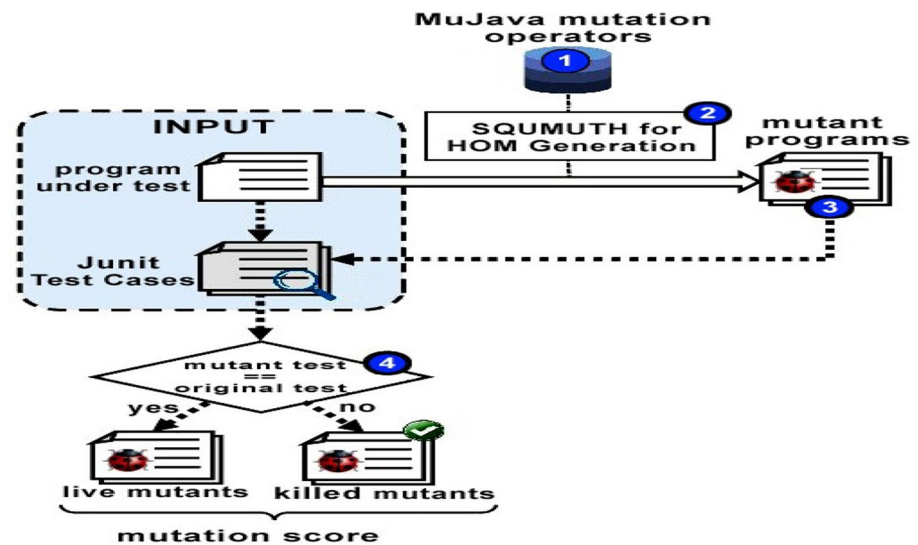
A higher S_{min} value promotes exploration, while a lower S_{min} value enhances the algorithm's exploitation capability. 't' signifies the current iteration value, and 'tm' represents the maximum iteration value. S_{min} plays a crucial role in balancing exploration and exploitation in SSA. Then a termination condition that incorporates reaching the maximum number of iterations (iter_max) or a satisfactory fitness threshold for achieving the desired coverage in mutation analysis. Final optimal solution: once the termination criterion is met, the algorithm ends. The mutants generated in the final optimal solution correspond to the location of the mutants on the hickory nut tree in the SSA context.

In general, to successfully implement the design of SQUMUTH, authors focused on critical components such as the encoding scheme, search strategies and mutation operators. The encoding scheme defines how mutant candidates are encoded and represented, allowing for the efficient manipulation of solutions. The search strategies in SSA used to balance exploration and exploitation, adapting dynamically based on the quality and diversity of the generated mutants, thereby ensuring efficient search and preventing premature convergence. Mutation operators are instrumental in creating HOMs that contribute significantly to the generation of diverse and high-quality mutants, thus improving the overall efficacy of the SSA.

Additionally, the proposed algorithm is highly influenced by the selection of parameter values. Hence, careful consideration of the algorithm's complexity and parameter tuning is crucial to improve the performance of SQUMUTH, as indicated in Table 3. By taking this into account, the SSA provides a robust SQUMUTH framework for generating HOMs, thereby enhancing the efficiency and effectiveness of mutation testing.

To enhance the comprehension of the proposed approach, the authors have presented a simplified pseudo-code representation of the design of SQUMUTH for HOMs generation as follows:

Fig. 1 The proposed framework for mutation score calculations



Input: Set of generated HOMs and Test suite

Output: List of strong mutants generated in the final optimal solution

Procedure SSA_HOM_Generation(HOMs, TestSuite)

InitializeAlgorithmParameters() # Step 1: Define SSA parameters (Table 3)

Mutants = GenerateRandomMutants() using Eq.(1) # Step 2: Initialize mutant population

While Not TerminationCriterionMet()

EvaluateRMRValues(Mutants, TestSuite) using Eq.(3) and Eq.(4) #Step 3: Compute fitness & RMR

SortMutantsByRMR (Mutants) # Step 4: Sort mutants by their RMR values.

If StrongMutantsExist() # Decision Point 1: Check if strong mutants exist.

MoveMutantsTowardsTrees(Mutants) using Eq.(5) # Step 5: SSA-driven movement

EndIf

If IntermediateMutantsExist() # Decision Point 2: Check if intermediate mutants exist.

MoveMutantsTowardsTrees(Mutants) using Eq.(6) # Step 6: SSA-driven movement

EndIf

If WeakMutantsExist() # Decision Point 3: Check if weak mutants exist.

ReplaceWeakMutants(Mutants) using Eq.(7) # Step 7: Replace weak mutants with new HOMs.

EndIf

If SeasonalMonitoringRequired() # Decision Point 4: Check if seasonal relocation is needed.

PerformSeasonalMonitoring() using Eq.(8), Eq.(9), Eq.(10) # Step 8: Prevent premature convergence

EndIf

EndWhile

Return GetFinalMutants(Mutants) # Step 9: Retrieve optimized strong mutants

EndProcedure

Pseudo code 1 SQUMUTH for HOM Generation

Figure 1 illustrates our framework for executing HOM generated by the SSA-based SQUMUTH approach. Initially, the original program is subjected to mutation operators provided by MuJava, which generate mutants. These mutants are then processed by the SQUMUTH algorithm to create HOMs, which involve combining multiple mutations. The idea is to introduce more complex errors into the program to challenge the robustness of the test cases. Once the HOMs are generated, the JUnit test cases are executed against them. The outcome of these tests determines whether the mutants are killed or remain live. The mutation score, a key metric in MT, is calculated based on the proportion of killed mutants to the total number generated, indicating the test cases' effectiveness in identifying potential errors in the program.

5 Experimental results and discussion

In this section, we outline the experiments conducted to assess the effectiveness of the proposed approach on well-known benchmark or subject programs.

The experiments were executed on Windows 11 machines featuring a 1.30 GHz Intel i5 CPU and 16 GB of memory. The proposed approach, implemented in Java, was seamlessly integrated into a mutation system to enhance the efficiency and efficacy of mutation analysis. To evaluate the effectiveness of our approach, we required two inputs: a set of mutants and test suites. For generate mutant inputs, we used the high-performing MT tool, MuJava, on Java programs [38]. This tool not only offers mutant generation capabilities but also provides essential resources such as environment setup guidelines, mutant generation procedures, test case setup instructions, and guidelines for running test cases against mutants. It enables researchers and testers to automatically generate FOMs, execute test suites against mutants, and generate execution results. Furthermore, MuJava supports mutant generation for both traditional and object-oriented programming languages. To generate test suite inputs for testing both the original program and its mutants, we utilized JUnit [39].

The study incorporated eight various small to medium-sized Java programs, including Monopoly, Cli, Chess, and Validator, each accompanied by comprehensive test suites assessed by developers [40]. Additionally, a triangle program, widely recognized as a benchmark for MT, was also utilized [41]. These benchmark programs were selected because they vary in size, complexity, and structure, helping to assess mutation testing from different perspectives. Their selection is based on previous studies [40, 41], which showed that these programs work well for mutation analysis. They also cover different types of software faults, making it easier to test how well mutation operators accomplish. Statistical data for our subject systems, presented in Table 4, aligns with prior studies [40], with minor variations likely due to the use of different mutation operators.

The dataset generated for the subject programs, as depicted in Table 5, provides a detailed breakdown of the mutants and the accompanying test suite produced for each program used in previous studies. This dataset offers valuable insights into the distribution of FOMs and HOMs, along with the percentages of executable, equivalent, and exceptional mutants across the programs. For instance, in the Validator program, out of 3982 mutants generated, 86.57% were executable, with 6.99% considered equivalent mutants and 4.9% exceptional mutants. Analyzing this dataset allows for a thorough examination of the effectiveness and characteristics of the mutants generated, helping

Table 4 Subject programs used in an experiment

Subject	LOC	FOMs (%used)	Mutation score
Validator	7563	1941 (97%)	36% (68%)
Chess	4754	956 (26%)	81% (86%)
Monopoly	4173	366 (90%)	80% (83%)
Cli	1585	249 (51%)	71% (81%)
Triangle	19	128 (100%)	92% (92%)
Ant	108,622	18,280 (92%)	57% (94%)
Math	104,506	103,663 (100%)	66% (71%)
JFreeChart	90,481	36,307 (99%)	21% (45%)

Table 5 Mutant dataset generated for subject programs

Subject program	FOM	HOM	Executable mutants (%)	Equivalent mutants (%)	Exceptional mutants (%)
Validator	3982	1327	86.57	6.99	4.9
Chess	981	490	91.53	2.5	0.7
Monopoly	543	181	89.67	8.65	0.4
Cli	234	78	88.11	0	0.7
Triangle	435	145	89.58	6.5	0.3
Ant	9851	4925	84.62	7.07	0.5
Math	7897	3948	90.56	10.93	4.3
JFreeChart	6541	3270	88.27	2.54	1

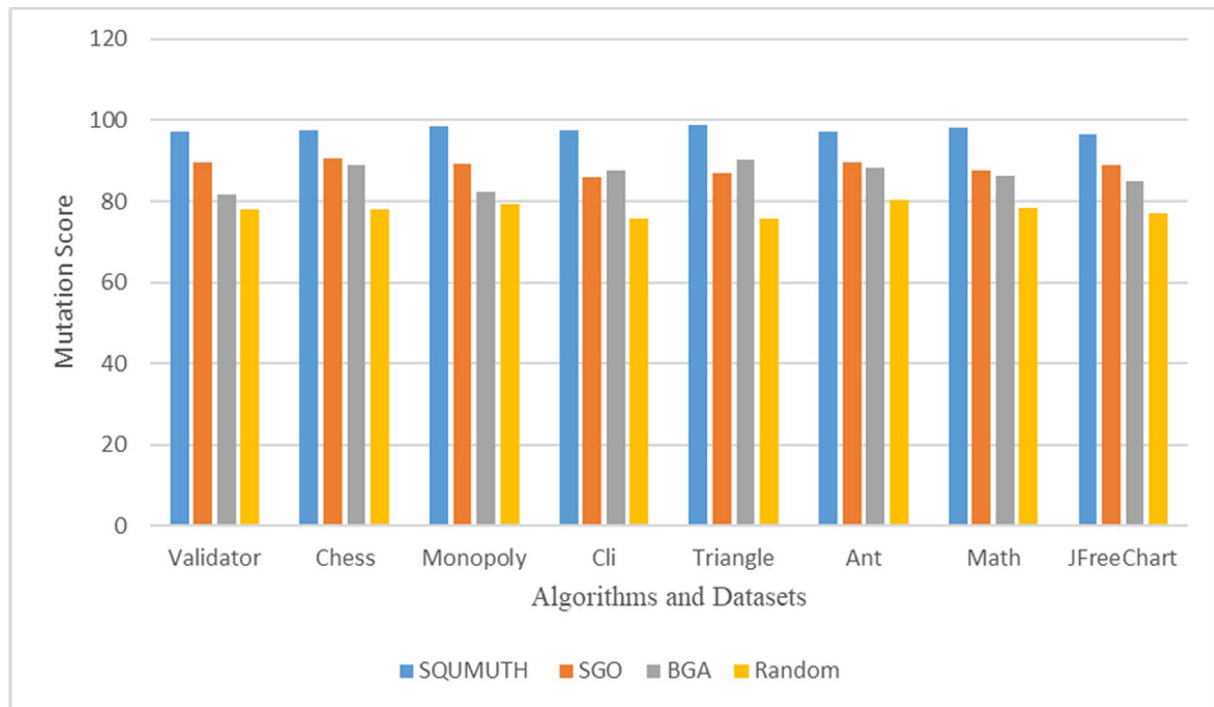


Fig. 2 Mutation score of SQUMUTH vs other 3 approaches

Table 6 Mutation score of SQUMUTH compared to 3 other algorithms

Subject Program	SQUMUTH	SGO	BGA	Random
Validator	97.27272727	89.5959596	81.61616162	77.87878788
Chess	97.47474747	90.70707071	88.98989899	78.08080808
Monopoly	98.48484848	89.19191919	82.22222222	79.49494949
Cli	97.57575758	85.95959596	87.67676768	75.85858586
Triangle	98.68686869	86.96969697	90.4040404	75.75757576
Ant	97.17171717	89.5959596	88.28282828	80.4040404
Math	98.18181818	87.67676768	86.16161616	78.28282828
JFreeChart	96.56565657	88.78787879	84.84848485	76.96969697

in the evaluation of the proposed approaches and the performance of previously proposed state-of-the-art methods in mutation analysis on given Java programs.

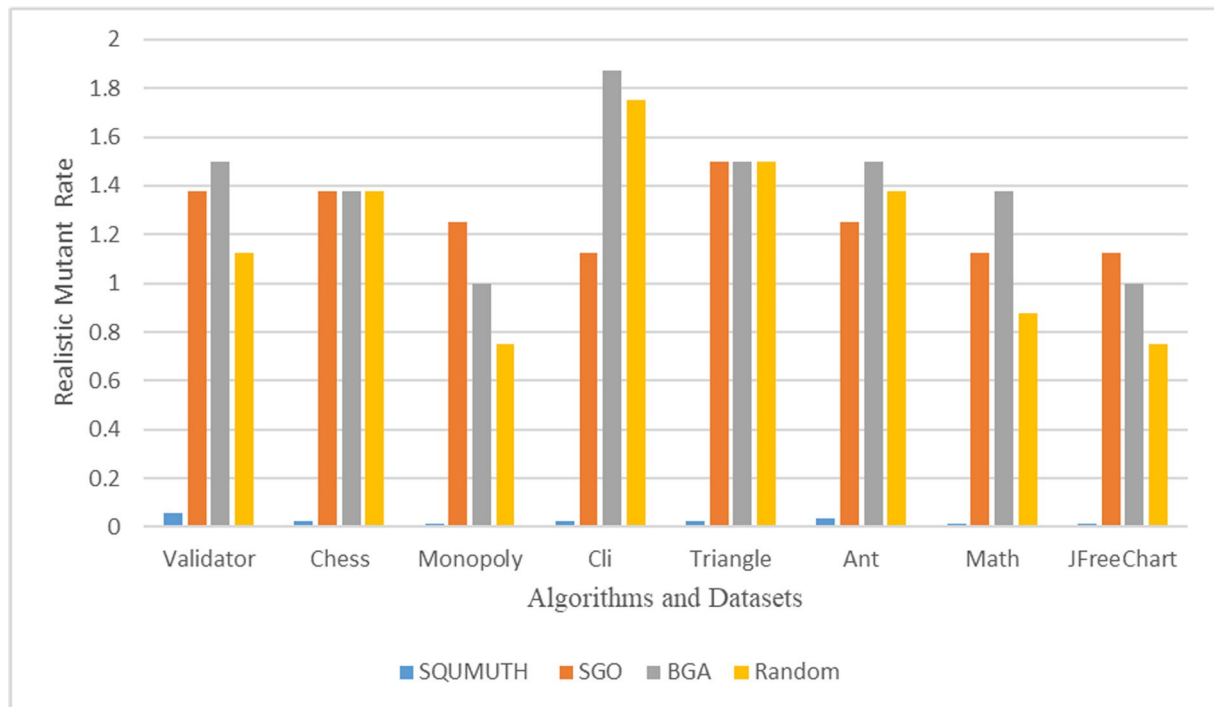
To evaluate the effectiveness of the proposed approach for HOM generation, two metrics are utilized to assess the performance and quality of the generated mutants. One key metric is the mutation score, which quantifies the efficacy of the test suite in detecting faults introduced by HOM. A higher mutation score indicates better fault detection capability, reflecting the quality of the mutants generated and the effectiveness of the testing approach and it serves as a crucial indicator of the test suite's efficacy in detecting faults introduced by HOM.

Across various subject programs, as illustrated in Fig. 2, SQUMUTH consistently outperforms other algorithms such as SGO, BGA, and Random. For instance, in the Validator program, SQUMUTH achieves a mutation score of 97.27%, exceeding SGO (89.60%), BGA (81.62%), and Random (77.88%), as detailed in Table 6. Overall, SQUMUTH consistently achieves higher mutation scores across all subject programs, indicating that it is more effective in identifying and eliminating mutants compared to the other testing methods. Additionally, SQUMUTH offers better test coverage, robustness, and enhanced fault detection capabilities, highlighting its potential as a preferred choice for HOM generation and establishing it as a more reliable approach for MT.

In addition to the mutation score, the resilience of mutants against the test suite is assessed using RMR values evaluation metrics, which provide insight into the ease of detecting and eliminating mutants, thus indicating the effectiveness

Table 7 Realistic mutant rate of all approaches

Subject Program	No of HOM	SQUMUTH	SGO	BGA	Random
Validator	1327	0.05682	1.375	1.5	1.125
Chess	490	0.02273	1.375	1.375	1.375
Monopoly	181	0.01136	1.25	1	0.75
Cli	78	0.02273	1.125	1.875	1.75
Triangle	145	0.02273	1.5	1.5	1.5
Ant	4925	0.03409	1.25	1.5	1.375
Math	3948	0.01136	1.125	1.375	0.875
JFreeChart	3270	0.01136	1.125	1	0.75

**Fig. 3** Realistic mutant rate of SQUMUTH vs. other 3 approaches

of the testing process in identifying hard or realistic faults. Lower RMR values suggest to those mutants that are challenging to detect and eliminate through testing or indicate better effectiveness in eliminating these tough mutants.

Across various subject programs, the RMR for SQUMUTH are consistently lower compared to other algorithms such as SGO, BGA, and Random as shown in Table 7. For example, in the Validator program, SQUMUTH demonstrates an RMR of 0.05682, indicating it is highly effective at killing difficult mutants or mutants that are relatively easier to detect compared to SGO (1.375), BGA (1.5), and Random (1.125). Overall, the data in presented Fig. 3 reveals that SQUMUTH consistently outperforms other approaches in reducing the number of hard-to-kill mutants across all subject programs. This suggests that SQUMUTH is highly effective and robust in eliminating even the most challenging mutants, which translates to a more comprehensive and reliable testing process. These findings emphasize the robustness and reliability of the SQUMUTH in enhancing the resilience of mutants against the test suite, thus contributing to improved fault detection and software quality assurance.

5.1 Statistical analysis of the mutation score

An ANOVA test on mutation score (Table 4) is conducted to statistically prove that SQUMUTH's mutation score is better compared to the other three algorithms (SGO, BGA, and Random). Null and Alternative hypotheses are framed to determine which specific algorithm has significantly better performance.

Table 8 ANOVA test on mutation score of table

Source of variation	Sum of squares (SS)	Degrees of freedom (df)	Mean square (MS)	F-statistic	p-value
Between Groups	1595.666	3	531.889	135.28	9.15E−17
Within Groups	110.093	28	3.932	–	–
Total	1705.76	31	–	–	–

- Null Hypothesis (H₀): There is no significant difference in the mean mutation scores among the algorithms (SQUMUTH, SGO, BGA, Random).

$$H_0 : \mu_{\text{SQUMUTH}} = \mu_{\text{SGO}} = \mu_{\text{BGA}} = \mu_{\text{Random}}$$

- Alternative Hypothesis (H_A): At least one algorithm has a mean mutation score that is significantly different from the others.

$$H_A : \text{At least one } \mu \text{ is different}$$

The F-statistic of 135.275 and the extremely low p-value confirm that there is a significant difference in mutation scores among the algorithms (Table 8). Given that the p-value is extremely low (much lower than the typical alpha level of 0.05), rejects the null hypothesis. This means that there is a statistically significant difference in the mean mutation scores among the algorithms. To determine which specific algorithm is better, A post-hoc test, such as Tukey's Honest Significant Difference (HSD) test is performed to determine which specific algorithm is better. It compares the means of each pair of groups. Based on the summary data it is concluded that SQUMUTH consistently has higher mean mutation scores across all subject programs compared to SGO, BGA, and Random. ANOVA test results on mutation score (Table 4), SQUMUTH appears to be the better algorithm in terms of mutation scores.

6 Threats to validity

The validity of our findings may be influenced by several factors. Externally, our results are limited by the specific subject programs, and testing environments selected for this study. While we aimed to reduce bias by using commonly referenced systems, the necessity to exclude certain tests due to technical challenges such as memory issues or infinite loops may affect the generalizability of our results. Internally, potential errors in our implementation and measurement processes could also impact the outcomes, despite our efforts to verify correctness through cross-validation and manual checks. Additionally, variations in the interpretation and application of metrics used to assess MT effectiveness may introduce construct validity concerns. Although we averaged results across multiple runs to mitigate the effects of nondeterminism, the inherent complexity of the systems studied, and the subjectivity involved in manual verification could still affect our conclusions.

7 Conclusion and future work

In this study, the authors introduced SQUMUTH, a novel algorithm that employs the SSA for the generation of HOMs in MT. The proposed approach has been rigorously evaluated and compared with state-of-the-art methods, including the SGO and BGA. The experimental results demonstrated that SQUMUTH outperforms these traditional methods across multiple benchmark programs. Specifically, SQUMUTH consistently achieved a higher mutation score, indicating its superior fault detection capabilities. Additionally, the RMR of SQUMUTH was significantly lower than that of its counterparts, underscoring the approach's potential efficiency in generating more challenging and realistic mutants. This enhancement improves the quality and reliability of software testing by boosting fault detection capability and resilience against difficult mutants. Furthermore, the study highlights the importance of adopting sophisticated mutation analysis techniques to address the evolving complexities of modern software systems.

The comparison with existing state-of-the-art methods highlights the advantages of our approach. Unlike the SGO and BGA, which may struggle with certain types of mutants or require more computational resources, SQUMUTH's use of SSA allows for a more balanced and effective exploration of the mutation space. This leads to better overall performance in detecting faults that might otherwise go unnoticed. To come to the point, SQUMUTH not only advances the field of MT by introducing a new method for HOM generation but also sets a new benchmark for comparison with other algorithms. The results of this study suggest that SQUMUTH could be a valuable addition to the MT toolkit, offering both improved performance and efficiency.

As for future work, further investigation could focus on refining the SQUMUTH to optimize its performance on larger and more complex software systems. Additionally, exploring the applicability of the approach to different programming languages beyond Java could broaden its scope and utility. Furthermore, integrating advanced machine learning techniques or evolutionary algorithms into the mutation analysis process may offer avenues for enhancing the efficiency and effectiveness of mutant generation and test suite evaluation. Overall, ongoing research in this area holds promise for advancing state-of-the-art in mutation testing and contributing to more resilient and reliable software systems.

Author contributions Subhasish Mohanty, Aliazar Deneke Deferisha, and Seifu Detso Bejo were responsible for performing the experiments and simulations, as well as analyzing the collected data. Sudhir Kumar Mohapatra contributed by developing the methodological framework and guiding the analysis process. Seifu Detso Bejo provided critical insights during data interpretation and assisted in refining the manuscript. Jyotirmaya Mishra played a key role in editing the manuscript and ensured that the final version adhered to publication standards. All authors reviewed the results, discussed their implications, and contributed to the preparation of the final manuscript.

Funding No funding is available.

Data availability Data will be provided on request.

Declarations

Ethics approval and consent to participate Not applicable.

Research involving human and/or animals No animal or human is involve in the experiment of this research.

Informed consent Not applicable.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Liu M, Fang S, Dong H, Xu C. Review of digital twin about concepts, technologies, and industrial applications. *J Manuf Syst.* 2021;58:346–61. <https://doi.org/10.1016/j.jmsy.2020.06.017>.
2. Forgács I, Kovács A. Software testing basics. In: *Modern software testing techniques*. Berkeley: Apress; 2024. p. 1–20. https://doi.org/10.1007/978-1-4842-9893-0_1.
3. Jamil MA, Nour MK, Alotaibi SS, Hussain MJ, Hussaini SM, Naseer A. Adaptive test suites generation for self-adaptive systems using SPEA2 algorithm. *Appl Sci.* 2023;13(20):11324. <https://doi.org/10.3390/app132011324>.
4. Sánchez AB, Parejo JA, Segura S, Durán A, Papadakis M. Mutation testing in practice: insights from open-source software developers. *IEEE Trans Software Eng.* 2024;50(5):1130–43. <https://doi.org/10.1109/TSE.2024.3377378>.
5. Kurtz B, Ammann P, Offutt J, Kurtz M. Are we there yet? How redundant and equivalent mutants affect determination of test completeness. In: *Twelfth IEEE Workshop on Mutation Analysis (Mutation 2016)* 2016; p. 1–6. <https://doi.org/10.1109/Mutation.2016.1>
6. Prado Lima JA, Vergilio SR. A systematic mapping study on higher order mutation testing. *J Syst Softw.* 2019;154:92–109. <https://doi.org/10.1016/j.jss.2019.04.031>.

7. Prado Lima JA, Vergilio SR. Search-based higher order mutation testing: A mapping study. In: Proceedings of the III Brazilian Symposium on Systematic and Automated Software Testing (SAST '18). Association for Computing Machinery, New York, NY. 2018, p. 87–96. <https://doi.org/10.1145/3266003.3266013>.
8. Shami TM, El-Saleh AA, Alswaitti M, Al-Tashi Q, Summakieh MA, Mirjalili S. Particle swarm optimization: a comprehensive survey. *IEEE Access*. 2022;10:10031–61. <https://doi.org/10.1109/access.2022.3142859>.
9. Katoch S, Chauhan SS, Kumar V. A review on genetic algorithm: past, present, and future. *Multimedia Tools Appl*. 2021;80(5):8091–126. <https://doi.org/10.1007/s11042-020-10139-6>.
10. Qin F, Zain AM, Zhou K-Q. Harmony search algorithm and related variants: a systematic review. *Swarm Evol Comput*. 2022;74: 101126. <https://doi.org/10.1016/j.swevo.2022.101126>.
11. Hernando L, Mendiburu A, Lozano JA. Hill-climbing algorithm: Let's go for a walk before finding the optimum. In: 2018 IEEE Congress on Evolutionary Computation (CEC), Rio de Janeiro, Brazil. 2018, p. 1–7. <https://doi.org/10.1109/CEC.2018.8477836>.
12. Jena JJ, Satapathy SC. A new adaptive tuned Social Group Optimization (SGO) algorithm with sigmoid-adaptive inertia weight for solving engineering design problems. *Multimedia Tools Appl*. 2024;83(1):3021–55. <https://doi.org/10.1007/s11042-021-11266-4>.
13. Shehab M, et al. A comprehensive review of bat-inspired algorithm: variants, applications, & hybridization. *Arch Comput Methods Eng*. 2023;30(2):765–97. <https://doi.org/10.1007/s11831-022-09817-5>.
14. Sharma SAMRS, Sharma VK. Research on use of nature-inspired algorithms in software testing. *Int J Innov Technol Explor Eng*. 2019;8(11):3446–52.
15. Jain M, Singh V, Rani A. A novel nature-inspired algorithm for optimization: squirrel search algorithm. *Swarm Evol Comput*. 2019;44:148–75.
16. Gay G, Salahirad A. How closely are common mutation operators coupled to real faults? In: 2023 IEEE Conference on Software Testing, Verification and Validation (ICST), 2023, p. 129–140. Dublin, Ireland. <https://doi.org/10.1109/ICST57152.2023.00021>.
17. Amare M, Mohapatra SK, Panigrahy TP. A systematic literature review on generating higher-order mutants. In: 2021 8th International Conference on Computing for Sustainable Global Development (INDIACom), 2021, pp. 131–136. New Delhi, India.
18. Sujeetha R, Akila K. Effective mutants' classification for mutation testing of smart contracts. In: Algorithms for intelligent systems. Springer Nature: Singapore; 2023. p. 263–79.
19. Dakhel AM, Nikanjam A, Majdinasab V, Khomh F, Desmarais MC. Effective test generation using pre-trained large language models and mutation testing. *Inf Softw Technol*. 2024;171: 107468. <https://doi.org/10.1016/j.infsof.2024.107468>.
20. Tamboon F, Khomh F, Antoniol G. A probabilistic framework for mutation testing in deep neural networks. *Inf Softw Technol*. 2023;155: 107129. <https://doi.org/10.1016/j.infsof.2022.107129>.
21. Langdon W, Clark D. Deep mutations have little impact. In: Proceedings of the 13th ACM/IEEE International Workshop on Genetic Improvement. p. 1–8. New York, NY, USA: ACM. 2024.
22. Abuljadayel A, Wedyan F. An approach for the generation of higher-order mutants using genetic algorithms. *Int J Intell Syst Appl*. 2018;10(1):34–45. <https://doi.org/10.5815/ijisa.2018.01.05>.
23. Wedyan F, Al-Shishani A, Jararweh Y. Gasubtle: a new genetic algorithm for generating subtle higher-order mutants. *Information*. 2022;13(7):327. <https://doi.org/10.3390/info13070327>.
24. Mohanty S et al. Generating higher-order mutants using PSO with Levy Flight (LPSO) algorithm. In: 2022 International Conference on Machine Learning, Computer Systems and Security (MLCSS). IEEE. 2022.
25. Lemieux C, Inala JP, Lahiri SK, Sen S. CodaMosa: escaping coverage plateaus in test generation with pre-trained large language models. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). 2023, p. 919–931. IEEE.
26. Bejo SD, Assefa BG, Mohapatra SK. BackIP: mutation-based test data generation using hybrid approach. In: 2021 International Conference on Information and Communication Technology for Development for Africa (ICT4DA). Bahir Dar, Ethiopia. 2021. p. 178–183. <https://doi.org/10.1109/ICT4DA53266.2021.9672216>.
27. Wong C-P et al. Efficiently finding higher-order mutants. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2020. <https://doi.org/10.1145/3368089.3409713>.
28. Oh S, Lee S, Yoo S. Effectively sampling higher-order mutants using causal effect. In: 2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW). 2021. <https://doi.org/10.1109/icstw52544.2021.00017>.
29. Ojdanic M, Garg A, Khanfir A, Degiovanni R, Papadakis M, Le Traon Y. Syntactic versus semantic similarity of artificial and real faults in mutation testing studies. *IEEE Trans Software Eng*. 2023;49(7):3922–38. <https://doi.org/10.1109/tse.2023.3277564>.
30. Nho V, Nguyen Q-V, Nguyen T-B. Toward improving the quality of mutation operator and test case effectiveness in higher-order mutation testing. *Vietnam J Comput Sci*. 2022;9(4):511–26. <https://doi.org/10.1142/s2196888822500282>.
31. Van Phol L, Binh NT, Parissis I. Higher-order mutant generator for Lustre programs. In: 2019 International Conference on System Science and Engineering (ICSSE). 2019. <https://doi.org/10.1109/icsse.2019.8823130>.
32. Pizzolo AV, Ferrari FC, Offutt J, Fernandes L, Ribeiro M. A systematic literature review of techniques and metrics to reduce the cost of mutation testing. *J Syst Softw*. 2019;157: 110388. <https://doi.org/10.1016/j.jss.2019.07.100>.
33. Liu J, Song L. Second-order mutation testing cost reduction based on mutant clustering using SOM neural network model. In: 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC), 2021, p. 974–979. Madrid, Spain. <https://doi.org/10.1109/COMPSAC51774.2021.000131>.
34. Sun C-A, Fu A, Guo X, Chen TY. ReMuSSE: a redundant mutant identification technique based on selective symbolic execution. *IEEE Trans Reliab*. 2022;71(1):415–28. <https://doi.org/10.1109/TR.2020.3011423>.
35. Do V-N, Nguyen Q-V, Nguyen T-B. Evaluating mutation operator and test case effectiveness by means of mutation testing. In: Lecture notes in computer science. Cham: Springer International Publishing; 2021. p. 837–50.
36. Rani S, Suri B. Equivalent mutant problem and its problem-solving techniques: a retrospective view. In: 2018 8th International Conference on Cloud Computing, Data Science & Engineering (Confluence); 2018, p. 791–797. Noida, India. <https://doi.org/10.1109/CONFLUENCE.2018.8442785>.
37. Diniz JP, Ferreira F, Ferrari F, Figueiredo E. Do mutations of strongly subsuming second-order mutants really mask each other? In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). 2023; pp. 114–124. Florence, Italy. <https://doi.org/10.1109/ISSRE59848.2023.00079>.

38. Khari M. Comparison and validation of mutation testing tools based on Java language. In: EAI/Springer innovations in communication and computing. Cham: Springer International Publishing; 2022. p. 13–29.
39. JUnit 5. (n.d.). The 5th major version of the programmer-friendly testing framework for Java and the JVM. Retrieved April 23, 2024, from <https://junit.org/junit5/>.
40. Harman M, Jia Y, Mateo PR, Polo M. Angels and monsters: An empirical investigation of potential test effectiveness and efficiency improvement from strongly subsuming higher-order mutation. In: Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14) (pp. 397–408). Vasteras, Sweden: ACM Press. 2014; <https://doi.org/10.1145/2642937.2643008>.
41. Jia Y, Harman M. Higher-order mutation testing. *Inf Softw Technol.* 2009;51(10):1379–93. <https://doi.org/10.1016/j.infsof.2009.04.016>.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.